

Approximation algorithms

Optimization problem $\mathcal{J}$

Instance $I \in \mathcal{J} \rightarrow c^*(I)$: optimal cost

A : algorithm for $\mathcal{J} \rightarrow c_A(I)$: cost of solution returned by A on I

Finding the optimal in an algorithm can be difficult or impossible, EX

- Vertex cover, 2-appx algorithm $c_{vcappx}(I) \leq 2c^*(I)$
- Partial information, like Skirental with D unknown, 2-appx algorithm $c_A(I) \leq 2c^*(I)$

Apx ratio is $\sup_{I \in \mathcal{J}} \frac{c_A(I)}{c^*(I)}$

Hitting Set

Given $T_1, T_2, \dots, T_p \subset S$

$$c : S \rightarrow R^+$$

Find $H \subset S$ that

$$\min \sum_{e \in H} c_e$$

$$\text{s.t. } H \cap T_j \neq \emptyset, j = 1, \dots, p$$

$$\alpha = \max |T_j|$$

Hitting Set is a generalization of Vertex Cover. Each edge is T set so we are finding $H \subset S$ that way $\exists h \in H, \forall T : h \in T$. Each vertex however has a cost of 1 because Hitting Set is a weighted generalization of Vertex Cover.

Hitting Set as ILP

$$\min \sum_{e \in S} c_e x_e$$

$$\text{s.t. } \sum_{e \in T_j} x_e \geq 1, (j = 1, \dots, p)$$

$$x_e \in \{0, 1\} \text{ and } 1 \text{ iff } e \in H$$

Linear Relaxation

$$\begin{aligned} \min \quad & \sum_{e \in S} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in T_j} x_e \geq 1, (j = 1, \dots, p) \\ & 0 \leq x_e \leq 1 \end{aligned}$$

Note that the $x_e \leq 1$ constraint is redundant because there is no optimal solution where $x_e > 1$ because the max an x is needed is 1 and all costs are positive so having a cost greater than 1 is useless.

Primal Dual Algorithm Idea

Primal $x^{(0)}$ (not necessarily feasible) and $y^{(0)}$ as a solution in dual that is feasible (note these are linked by comp slack)

We iterate forward to create $x^{(1)}, y^{(1)}$ where x is not necessarily feasible but we are going to reduce the infeasibility region from $x^{(0)}$ to $x^{(1)}$

Dual of Lin Relax

y_p for each T_p

So dual is

$$\begin{aligned} \max \quad & \sum_{j=1}^p y_j \\ \text{s.t.} \quad & \sum_{j: e \in T_j} y_j \leq c_e, (\forall e \in S) \\ & y_j \geq 0 \end{aligned}$$

Continue until x^* and y^* are both feasible and satisfy comp slack.

Thus optimal $c^* : \sum_{j=1}^p y_j \leq c^* \leq \sum_{e \in S} c_e x_e$

Comp Slack

$$x_e s_e = 0, (e \in S)$$

$$s_e = c_e - \sum_{j: e \in T_j} y_j \geq 0$$

$$\geq 0 \text{ for feasible } y$$

$$y_j (\sum_{e \in T_j} x_e - 1) = 0, (j = 1, \dots, p)$$

This is good but we also need to try and get an optimal integrality idea. For Successive Shortest Paths, we had integrality bc MCNF is unimodular so optimal solution is integer.

Primal Dual Algorithm for Hitting Set

We relax the comp slack constraint of

$$y_j (\sum_{e \in T_j} x_e - 1) = 0, (j = 1, \dots, p)$$

Each iteration we slowly introduce these set constraints.

Thus:

$$H = \emptyset$$

$$y_j = 0$$

```
while \exists j: T_j \cap H = \emptyset:
    increase y_j until v s.t. \exists e s_e = 0
    H.append(e)
return H
```

Invariants

1. $s_e \geq 0$
2. $y_j \geq 0$

True initially, (straight forward)

At each iteration we are increasing y_j to an $s_e \geq 0$

$\rightarrow y_j$ never decreases so $\rightarrow y_j \geq 0$

$\rightarrow s_e$ does not go below 0 because we no longer use that vertex

Thm HSPD Is α -apx algorithm for HS

Corollary HSPD is 2-apx for VC

Proof Cost of returned H :

$$c(H) = \sum_{e \in H} c_e = \sum_{e \in H} \sum_{j: e \in T_j} y_j = \sum_{j=1}^p |H \cap T_j| y_j \leq \alpha \sum_{j=1}^p y_j \leq \alpha c^*$$

$s_e = 0$ when e added to H and never change any of those sets again because they are satisfied ($y_j, j : e \in T_j$) (think comp slack)

The last is given from that the optimal solution is always $c^* \geq \max \sum y_j$

A dif thing idk he didnt name it

$$\max \prod_{i=1}^n (1 - x_i)$$

$$\text{s.t. } \sum_{i=1}^n x_i \geq \beta$$

$$0 \leq x_i \leq 1$$

Where β is a parameter where $0 \leq \beta \leq 1$

Claim WLOG opt $x_i^* \leq \beta$ otherwise we can set a single x_i to β which results in a objective of $1 - \beta$. Since we are assuming that $\exists x_i^* \in x^*, x_i^* > \beta$ then the objective would be smaller than this.

Thm $x_i^* = \frac{\beta}{n}$ as optimal solution

Proof by induction: $n = 1$ trivial

inductive step:

$$\max (1 - x_n) \prod_{i=1}^{n-1} (1 - x_i)$$

$$\text{s.t. } x_n + \sum_{i=1}^{n-1} x_i \geq \beta$$

If we assume $\bar{x}_n$ is fixed then:

$$\max \prod_{i=1}^{n-1} (1 - x_i)$$

$$\text{s.t. } \sum_{i=1}^{n-1} x_i \geq \beta - \bar{x}_n$$

This is easy to solve because this is the same as our $n - 1$ induction so optimal is:

$$x_i^* = \frac{\beta - \bar{x}_n}{n-1} \text{ so the optimal value is } (1 - \frac{\beta - \bar{x}_n}{n-1})^{n-1}$$

We can take the derative of this with respect to $\bar{x}_n$ to determine the maximum of this function is. We find that the optimal $\bar{x}_n = \frac{\beta}{n}$, this makes the $x_i^* = \frac{\beta - \beta/n}{n-1} = \frac{\beta}{n}$

Thus the final cost is $c^* = (1 - \frac{\beta}{1})^n \leq e^{-\beta}$ which is equality as n approaches ∞ due to this being the definition of e

Algorithm HSRand1

$$\begin{aligned} \min & \sum_{e \in S} c_e x_e \\ \text{s.t. } & \sum_{e \in T_j} x_e \geq 1, (j = 1, \dots, p) \\ & 0 \leq x_e \leq 1 \end{aligned}$$

for each e in S do:
 $H = H \cup \{e\}$ w/ prob x_e^*

$E(c(H)) = \sum_{e \in S} c_e \Pr[e \in H] = \sum_{e \in S} c_e x_e^* \leq c^*$ better than optimal but may not give a cover

Thus we find the probability that a set is not hit is:

$$\Pr[e \cap H = \phi] = \Pr[a \notin H] * \Pr[b \notin H] = (1 - x_a^*) * (1 - x_b^*) = 1/4 \rightarrow \text{vertex cover case}$$

For number of edges not covered:

$$E[\# \text{edges not covered}] = E[\sum_{j=1}^p x_j] \sum_{j=1}^p E[x_j] = \sum_{j=1}^p \Pr[x_j \cap H = \phi]$$

Where $x_i = 1$ iff $T_i \cap H = \phi$ else 0.

Thus in the case of $m = 3$ edges we have the expected number of edges covered as $\frac{3}{4}$

$\Pr[T_j \cap H = \phi] = \prod_{e \in T_j} \Pr[e \notin H] = \prod_{e \in T_j} (1 - x_e^*) \leq 1$ from before we have a better bound as this probability being less than $\frac{1}{e}$

Thus $E[\#T_j \text{ not covered}] = \frac{p}{e}$

Algorithm HSRand2

Solve LR $\rightarrow x_e^*$
for $h=1$ to t do:
 $H \leftarrow H \cup \{e\}$ w/ prob x_e^*

$$\Pr[T_j \text{ nohit}] \leq \frac{1}{e^t}$$

$$E[\# \text{sets nohit}] \leq \frac{p}{e^t}$$

Thus $E[c(H)] \leq t \times c(H^*)$

HSR vs PD

Type	Apx Ratio	Failute
PD	$\max T_j $	NA
HSR	$\ln p$	$E[nothit] \leq 1$

MAX SAT

Clauses are similar to SAT, conjunctive normal form, so variables joined by OR's
goal is to maximize the number of true clauses.

2-apx for MAX SAT

Randomly select true or false for each x with probability $1/2$

So probability a clause is satisfied is $1 - \frac{1}{2^k} \geq \frac{1}{2}$ where k is number of literals in MAX SAT

So expected # of satisfied clauses is $\geq \frac{p}{2}$

Alternatively set all to True and all to False and pick the larger of the two.

Algorithm MAXSAT Rand

```
for each variable x do:  
    set x to true w/ prob 0.5
```

If a clause C_j contains k literals:

$$Pr[C_j sat] = 1 - \frac{1}{2^k} \geq \frac{1}{2}$$

So expected number of satisfied clauses is $\frac{p}{2} \geq \frac{opt}{2}$

Thus MAXSAT Rand is a 2-apx algorithm for MAX SAT

Max SAT as ILP

There are p clauses. Let set of positive literals be C_j^+ and negatives for a clause is C_j^-

$$\max \sum z_j$$

$$\text{s.t. } \sum_{x_i \in C_j^+} x_i + \sum_{x_i \in C_j^-} (1 - x_i) \geq z_j, j = 1, \dots, p$$

$$x \in \{0, 1\}$$

$$z \in \{0, 1\}$$

MAX SAT Lin Relax

Thm this is a $(1 - \frac{1}{e})$ -appx

$$Pr[C_j \text{ SAT}] = \prod_{i \in C_j^+} (1 - y_i^*)$$

$$\text{for simplicity } C_j^- = \phi$$

$$\text{s.t. } \sum y_i^* \geq z_j^*$$

worst case:

$$\leq \max \prod (1 - y_i^*) \rightarrow \leq (1 - \frac{z_j^*}{k})^k$$

$$\text{note } k = \text{num literals in } C_j$$

$$Pr[C_j \text{ SAT}] \geq 1 - (1 - \frac{z_j^*}{k})^k \geq (1 - \frac{1}{e}) z_j^*$$

$$E[\#SATC] \geq (1 - \frac{1}{e}) OPT$$

$$\text{NOTE DERIVATIVE OF } (1 - \frac{a}{b})^b = (1 - \frac{1}{b})^b a \geq (1 - \frac{1}{e}) a$$

MAX SAT Mix

Find assignments from MAXSAT RND, MAXSAT Lin Relax and pick the best one (satisfies most clauses)

n_1 expected satisfied from MAXSAT RND and n_2 is the other

Thm This is $\frac{3}{4}$ - Appx

$$\text{So } E[\text{MAXSAT Mix}] = \max n_1, n_2 \geq E[\frac{n_1 + n_2}{2}] = \frac{1}{2} n_1 + \frac{1}{2} n_2$$

$$C^k = \{\text{Clauses } C_j : |C_j| = k\}$$

$$\text{Let } \alpha_k = 1 - \frac{1}{2^k}$$

And $\beta_k = (1 - (1 - \frac{1}{k})^k)$

PF

$$\begin{aligned}
E[|C|SATbyMAXSATMix] &= E[\max\{n_1, n_2\}] \geq E[\frac{n_1+n_2}{2}] \\
&= \frac{1}{2}(E[n_1] + E[n_2]) \\
E[n_1] &= \sum_{k=1}^{\infty} \sum_{C_j \in C^k} \alpha_k \\
E[n_2] &= \sum_{k=1}^{\infty} \sum_{C_j \in C^k} \beta_k z_j^* \\
\frac{1}{2}(E[n_1] + E[n_2]) &\geq \sum_{k=1}^n \sum_{C_j \in C^k} \frac{\alpha_k + \beta_k}{2} z_j^* \geq \frac{3}{4} \sum z_j^* \geq \frac{3}{4} OPT
\end{aligned}$$

MAX CUT

Given complete undirected graph $G = (V, E)$ and weights $w_e = w_{ij}, \forall e = (i, j) \in E$

a_{ij} = distance between $i, j \in V \wedge i \neq j$

Find cut $(S, V - S)$ that $\max c(S, V - S) = \sum_{(i,j) \in \times(S, V-S)} \alpha_{ij}$

Essentially find a set of vertices S that maximizes the sum of all edges that intersect the cut line.

$\delta_e = 1$ iff $e \in \times(S, V - S)$ and 0 otherwise.

$$c(S, V - S) = \sum_{e \in E} a_e \delta_e$$

MAXCUT Rand

For each $v \in V$, add v to S with probability $\frac{1}{2}$

Claim This algo is a 2-approx

$$\textbf{Proof } E[c(S, V - S)] = \sum_{e \in E} a_e E[\delta_e] = \frac{1}{2} \sum_{e \in E} a_e \geq \frac{1}{2} OPT$$

Decision Vars:

$x_i = 1$ iff $i \in S$ and -1 otherwise, $(\forall i \in V)$.

$x_i x_j = 1$ iff same side and -1 iff opposite sides

$$\text{Thus } \frac{1 - x_i x_j}{2} = \delta_{ij}$$

Integer Quadratic program

$$\max c(S, V - S) = \max \sum_{e \in E} a_e \frac{1 - x_i x_j}{2}$$

s.t. $x_i \in \{-1, 1\}$

Instead of our usual relaxation, we instead relax x_i to a nd unit vector v_i

Relaxtion

$$\max c(S, V - S) \leq \max \sum_{e \in E} \alpha_e \frac{1 - v_i^T v_j}{2}$$

s.t. $v_i \in S_n(i \in V)$

B = matrix of the v

$X = B^T B$ is a semi definite program that we are trying to maximize.

Thus X is the matrix of dot products of these vectors

Diagonal entries are 1 because all v are part of unit hypersphere

If $v_i^T v_j = -1$ then wed want δ_e be 1 and conversly, $v_i^T v_j = 1$ then wed want $\delta_e = 0$

Now we need to generate a cut from the relaxation. So here are a few ideas (that don't really work):

1. Random Rounding
2. Vectors alr express clustering so more clustering may not be helpful
3. Project onto x axis (rotations are identical so projection would mess with results)
4. Almost anything to round would work

Random hyperplane through origin and that is used for random rounding.

$$E[c(S, V - S)] = E[\sum_{e \in E} a_e \delta_e] = \sum_e a_e Pr[\delta_e = 1]$$

$Pr[\delta_e = 1] = \frac{\alpha}{\pi}$ where α is the angle between the two vectors.

$$Pr[\delta_e = 1] = \frac{\arccos(v_i^T v_j)}{\pi}$$

$$E[c(S, V - S)] = E[\sum_{e \in E} a_e \delta_e] = \sum_e a_e Pr[\delta_e = 1] = \sum_e a_e \frac{\arccos(v_i^T v_j)}{\pi}$$

$$f(z) = \frac{\arccos(z)}{\pi} \frac{z}{1-z}$$

Applying derivative to find that $\hat{z} = \sqrt{\frac{1-\hat{z}}{1+\hat{z}}}$

$$f(z) = \frac{\arccos(z)}{\pi} \frac{z}{1-z} \rightarrow f(z) \geq f(\hat{z}) = 0.879$$

$$= \sum_e a_e \frac{\arccos(v_i^T v_j)}{\pi} \geq 0.879 \sum_e a_e \frac{1 - v_i^T v_j}{2} \geq 0.879 c^*$$